

Implementation Conflict and Correction Brief

Smoothed Slices / Slices Perspective / NF-iSAM MATLAB Project

Detailed handoff document for a code-editing AI or developer

Scope: static source-level review of the current project archive under /mnt/data/proj_work. I did not run MATLAB, so this document identifies implementation departures and exact places to modify, but it does not certify numerical correctness after edits.

Main rule for the next AI: do not rewrite the project from scratch. Make minimal, modular edits; preserve the shared result contract and existing directory structure; add explicit provenance/warnings wherever an implementation intentionally departs from the papers.

Prepared: 2026-08-17 08:10

1. Executive Summary

The current implementation is scientifically promising and reasonably modular, but it must not be described as an exact reproduction of all paper methods. The cleanest implementation story is:

- Slices: paper-faithful only for the controlled low-dimensional Eq. (16)-type demonstrator; the general graph engine is an implementation generalization based on explicit proposal-density weighting and finite support lookup.
- Smoothed Slices: a proposed extension, not part of the Slices paper; current code implements finite-support Mode A and a one-level conditional-smoothing replacement in the Eq. (16)-type path, while the general graph path uses active-set sparsification rather than the full recursive R_r tower.
- NF-iSAM: conceptually follows Bayes-tree clique conditional samplers, but the current MATLAB version rebuilds the tree, reuses by clique identity, propagates child separator information through samples/resampling, and trains a custom MATLAB RQ-spline flow with a reduced iteration cap.

The main required correction is not only code. The AI must also correct labels, provenance, status flags, method cards, report text, and diagnostic warnings so that the app never overclaims paper fidelity.

2. Paper-Faithfulness Standard to Enforce

Use this standard before changing code. A method is paper-faithful only if its target mathematical object, proposal/sampling mechanism, recursion/update rule, and output semantics match the paper. If one of those differs, mark it explicitly as an implementation approximation or proposed extension.

For the Slices Perspective paper, the relevant paper objects are:

- Forward factorization: $P(\omega_j, S_j | D_j) = P(\omega_j | S_j, D_j) f_{\text{new}}(S_j | D_j)$.
- Generated separator factor: $f_{\text{new}}(S_j | D_j)$ is obtained by marginalizing the local joint over ω_j .
- Monte Carlo slice estimator: sample ω_j from a factor f_{prime} in $F_{\{j-1\}}(\omega_j)$, exclude that sampling factor from the product, and evaluate the remaining factors as slices.
- Example around Eq. (16): after eliminating x_i and then l , the new factor includes the nested-looking integral $\int f(x_i^{(n)}, l) f(l, x_j) dl$.
- The paper message is direct use of slices, not KDE or learned density reconstruction for generated factors.

For NF-iSAM, the relevant paper objects are:

- Bayes-tree factorization into clique conditionals $p(F_C | S_C, z_C)$.

- Bottom-up training of clique conditional samplers using normalizing flows.
- Propagation of separator density information upward through the Bayes tree.
- Top-down/root-to-leaf ancestral sampling of the full joint posterior.
- Published implementation details include rational-quadratic spline flows, 2000 training samples, 9 spline segments, hidden units of 8, and a stopping criterion based on relative loss flattening over two 50-iteration windows.

3. Highest-Priority Fix List

Priority	Issue	Required change	Files to touch
P0	Do not overclaim fidelity	Change comments and UI/report wording that say Slices is “reproduced faithfully” without qualifying the engine. Add result-level deviation flags.	runSlicesMethod.m, methodCard.m, diagnosticsReport.m, report text
P0	Mark engines separately	Add result.implementation.engineFidelity = paper_demo general_importance_sampling proposed_extension prototype.	runInferenceCore.m, runGeneralCore.m, runNFISSAMMethod.m
P0	Smoothed general path	Either implement true recursive R_r in the general engine or rename the current behavior as active-set sparse generated-factor approximation.	forwardEliminationGeneral.m, runSmoothedSlicesMethod.m, methodCard.m
P0	NF-iSAM reuse and child separators	Expose rebuilt-tree reuse and sample-based separator propagation as deviations, not exact paper reproduction.	incrementalUpdate.m, combineChildSeparators.m, diagnosticsReport.m
P1	1-D separator limitation	Keep if needed, but report explicitly; do not imply general high-dimensional separator support in the Eq. (16) engine.	estimateNewFactor.m, README, report
P1	Training cap	Report MaxIterations=200 as runtime cap and not paper setting. Add warning when stoppedBy == max_iter.	commonMethodConfig.m, trainFlow.m, runNFISSAMMethod.m

4. Slices: Conflicts, Limitations, and Required Fixes

Overall status: the small Eq. (16)-type engine is close to the Slices paper mechanism. The general graph engine is not paper-literal; it is an importance-sampling/tabulated-support implementation that should be labeled accordingly.

Issue	Location	Why this conflicts / limits paper fidelity	What the AI should change
Overclaim in wrapper comment	runSlicesMethod.m:L1-L20	The comment says “reproduced faithfully” and “No KDE and no flow appear anywhere in this path.” This is accurate for the low-dimensional route but too broad once the same public method can dispatch to the general engine.	Change to: “paper-faithful for the controlled Slices demonstrator; general graph mode uses a documented implementation generalization.” Add result fidelity tag.
1-D separator restriction	estimateNewFactor.m around the Lemma-1 branch	The Slices paper uses general separators S_j . The current Lemma-1 route errors unless $\text{numel}(\text{separator})==1$. This is fine for x_{-i-l-x_j} , but not general high-dimensional paper fidelity.	Do not remove without implementing vector separator support. Add explicit diagnostic field and UI/report warning: scalar Lemma-1 engine only.
General engine uses explicit proposal-density division	forwardEliminationGeneral.m:L16-L25 and proposal code	The general engine keeps all removed factors and divides by $q(\omega)$. That differs from the paper Eq. (13) style where the sampling factor is selected from F and excluded from the product with normalizer handling.	Label as importance-sampling generalization. If paper-faithful mode is desired, add a separate route that samples from f prime and excludes it exactly.
Finite support / lookup of generated factors	ApproximateFactor.m tabulated mode and general engine support handling	The Slices paper avoids intermediate KDE/learning reconstruction. The code stores finite tabulated supports and may answer by nearest-support lookup. This is a practical approximation and can bias high-dimensional separators.	Add warning fields: lookupDistanceMean/Max already exist in parts; surface them in diagnostics and exported report. Consider exactGeneratedFactors mode for small tests.
Backward pass is low-dimensional/grid-oriented	backwardMarginals.m and general backward sampler	The backward pass is acceptable for demos but should not be described as a fully general high-dimensional paper reproduction.	Document as “demo backward sampler.” Add high-dimensional reference tests before claiming generality.
MMD early stopping breaks joint row consistency when copying old samples	backwardSampleGeneral.m early-stopping comments	Paper early stopping is a heuristic for stopping propagation. Current reuse can copy old samples and break row-wise joint correlation across the stopping boundary.	Keep the heuristic, but add result.warning = joint_sample_rows_not_preserved_when_early_stopped. In plots, treat marginals as valid diagnostics and joint samples with caution.

Relevant current code excerpt:

runSlicesMethod.m: wrapper currently overstates paper fidelity

```

1: function result = runSlicesMethod(caseData, config)
2: %RUNSLICESMETHOD The Slices Perspective method, reproduced faithfully.
3: %   RESULT = RUNSLICESMETHOD(CASEDATA, CONFIG) runs Algorithms S1-S4 with the
4: %   paper's nested inner estimator, Eq. (23).

```

Static source-level handoff; verify with MATLAB tests after edits.

```

5: %
6: %   No KDE and no flow appear anywhere in this path: f_new is a mixture of
7: %   slices, the conditional is a ratio of factor products, and the backward
8: %   marginals are mixtures of conditional slices. The Slices spec section 13
9: %   lists each of those as a "do not do" if replaced by a reconstruction, and
10: %   the whole point of the comparison is that this method does not
11: %   reconstruct anything.
12:
13: arguments
14:     caseData (1,1) struct
15:     config (1,1) struct = methods.commonMethodConfig()
16: end
17:
18: config.innerEstimator = "nested";
19: result = methods.runInferenceCore("Slices", caseData, config);
20: end

```

runInferenceCore.m: two engines under one entry point

```

19: % Two engines, one entry point. The three-node engine follows Algorithm S1
20: % literally and is validated against dense quadrature; the general engine
21: % handles graphs of any shape by dividing out an explicit proposal density.
22: % The case says which one it needs, because the choice is a property of the
23: % graph rather than of the method, and both methods must take the same route
24: % for the comparison between them to mean anything.
25: if isfield(caseData, 'engine') && caseData.engine == "general"
26:     result = methods.runGeneralCore(methodName, caseData, config);
27:     return
28: end

```

estimateNewFactor.m: Lemma-1 branch where scalar separator and nested/RCS switch occur

```

70:     'Counter', graph.Counter, ...
71:     'Diagnostics', struct('route', "unary", 'numSamples', N));
72:
73:     info.numOuterSamples = N;
74:     info.samplingRoute    = "unary factor " + route.factor.Name;
75:
76:     % =====
77:     case "lemmal"
78:     % =====
79:         af = route.approximate;
80:
81:         if numel(separator) ~= 1
82:             error('methods:slices:multiDimSeparator', ...
83:                 ['The Lemma-1 route currently supports a 1-D final separator; ' ...
84:                 'S_j = {%s}. Multi-dimensional separators arrive with the ' ...
85:                 '2-D SLAM case study.'], strjoin(cellstr(separator), ','));
86:         end
87:         sepVar = separator(1);
88:
89:         % Finite separator support S, |S| = R_s.
90:         S = methods.slices.separatorSupport(graph, sepVar, config);
91:         Rs = numel(S);
92:
93:         % Decompose the removed factors into b_0, g_0 and a.
94:         g0 = route.factor; % xi_0 -> omega
95:         front = af.sliceFactorsExcluding(omega); % b_0(xi_0, s)
96:         fuse = route.others; % a(omega, s)
97:
98:         xi0 = af.Samples; % X_0, |X_0| = N_0
99:         N0 = numel(xi0);
100:
101:         % b_0(xi_0^(n), s) : |X_0| x |S|
102:         B = ones(N0, 1);
103:         for i = 1:numel(front)
104:             B = B .* front(i).evaluate(struct( ...
105:                 matlab.lang.makeValidName(af.EliminatedVar), xi0, ...

```

Smoothed Slices Project — Implementation Conflict and Correction Brief

```

106:         matlab.lang.makeValidName(sepVar), S));
107:     end
108:
109:     % R(xi_0^(n), s) : |X_0| x |S|, by the selected inner estimator.
110:     switch string(config.innerEstimator)
111:     case "nested"
112:         [R, innerInfo] = methods.slices.innerNestedEstimate( ...
113:             af, g0, fuse, omega, sepVar, S, config);
114:     case "rcs"
115:         [R, innerInfo] = methods.smoothed.evaluateSurfaceRecursion( ...
116:             af, g0, fuse, omega, sepVar, S, config);
117:     otherwise
118:         error('methods:slices:unknownInnerEstimator', ...
119:             'Unknown innerEstimator "%s".', config.innerEstimator);
120:     end

```

5. Smoothed Slices: Conflicts, Limitations, and Required Fixes

Overall status: Smoothed Slices is correctly marked in several places as a proposed extension. The implementation is useful, but it currently realizes only a subset of the theory: finite-support Mode A, mostly one-level Eq. (16)-style recursion, and active-set sparsification in the general graph path.

Issue	Location	Why this conflicts / limits theory	What the AI should change
Not a published Slices method	runSmoothedSlicesMethod.m:L1-L17, L53-L58	This is not a conflict if labelled clearly. It becomes a conflict only if UI/report text implies it is in the Slices paper.	Keep provenance.notInPaper=true everywhere. Add "proposed extension" to exported diagnostics and figure captions.
Only finite-support Mode A implemented	evaluateSurfaceRecursion.m:L49-L53	The Smoothed Slices theory includes finite/basis/regression modes, but code errors for basis/regression.	Do not claim Modes B/C. Add TODO or implement fitSurfaceBasis and regression path before presenting them as available.
One-level recursion in main Eq. (16) path	evaluateSurfaceRecursion.m entire function	The function builds X_1 , terminal R_1 , P_0 , and returns R_0 . That matches $x_i \rightarrow x_j$, but not arbitrary-depth R_r tower.	Rename diagnostic to "one_level_finite_rcs" or implement recursive loop over $r=H-1, \dots, 0$ for longer paths.
General engine not true R_r tower	forwardEliminationGeneral.m active-set branch	For general graphs, Smoothed differs mainly by sparse active-set filtering of the slice matrix; this is not the same as evaluateSurfaceRecursion.	Either implement RCS in the general engine or call it "active-set sparse Slices / Smoothed-lite general mode."
Pooled and quantile-thinned support is implementation choice	buildSamplingPath.m:L1-L91	Support construction is not specified in either paper; it approximates the conditional integral with shared support and Voronoi weights.	Keep it, but record it as an approximation. Export supportInfo with pooling rule, support size, nodeWeights, and thinning rule.
Sparse active-set recursion is truncated expectation	buildActiveSuccessors.m:L1-L96	Renormalizing over K retained successors changes the expectation unless retained mass is near one or activeSetSize is dense.	Report retainedMassMin/Mean. Warn if retained mass below threshold. Do not call sparse RCS exact.
Cost model wording should be precise	evaluateSurfaceRecursion.m info struct	Cost should be described as $B_0^*K^*R_s$ for the current one-level implementation and $\text{sum}_r B_r^*K_r^*R_s$ for recursive theory. Avoid ambiguous labels like $ X_0 N_0 S $.	Update costModel strings and exported cardinality text.

Relevant current code excerpt:

runSmoothedSlicesMethod.m: RCS switch and provenance

```

24: config.innerEstimator = "rcs";
25: % Recorded, not just done. The Compare Methods tab reads the budgets back off
26: % each result and flags any that disagree; this one is MEANT to disagree, and
27: % a difference with no reason attached is exactly the case that panel exists
28: % to catch.
29: config.provenance.innerEstimator = ...
30:     "set to rcs by runSmoothedSlicesMethod: Eq. (50) in place of the nested " + ...
31:     "Eq. (23). This single field is the whole difference between this method " + ...
32:     "and Slices, which is why both run through one driver.";
33:
34: % On a general graph the online gain has to come from the active successor
35: % sets, since there is no nested inner sampling left to remove: the general
36: % engine already does the inner integral as one slice average. Leaving
37: % activeSetSize at Inf there would select the dense update of Eq. (48) and
38: % make this method bit-for-bit identical to Slices -- two robots drawn on top
39: % of each other and a comparison with nothing in it. A default is therefore
40: % chosen from the support budget, and it is recorded in the result so the
41: % number is never mistaken for something the user asked for.

```

Static source-level handoff; verify with MATLAB tests after edits.

Smoothed Slices Project — Implementation Conflict and Correction Brief

```

42: if isfield(caseData, 'engine') && caseData.engine == "general" ...
43:     && ~isfinite(config.activeSetSize)
44:     config.activeSetSize = max(8, round(config.activeSetFraction * ...
45:         config.separatorSupportSize));
46:     config.provenance.activeSetSize = sprintf( ...
47:         "chosen by runSmoothedSlicesMethod as %g of |S| for the general engine", ...
48:         config.activeSetFraction);
49: end
50:
51: result = methods.runInferenceCore("Smoothed Slices", caseData, config);
52:
53: result.provenance = struct( ...
54:     'origin',      "proposed extension", ...
55:     'basedOn',     "Slices Perspective (Shienman et al., IROS 2024)", ...
56:     'newContent',  "tower-property reinterpretation; cached conditional smoothing surfaces R_r;
finite-cardinality recursion with active successor sets", ...
57:     'notInPaper', true, ...
58:     'uiNote',     "Smoothed Slices is our proposed extension, not an algorithm published by the
Slices authors");

```

evaluateSurfaceRecursion.m: finite one-level conditional-smoothing surface implementation

```

1: function [R0, info] = evaluateSurfaceRecursion(af, g0, fuse, omega, sepVar, S, config)
2: %EVALUATESURFACERECURSION Smoothed Slices inner estimator, Eqs. (48)-(53).
3: % [R0, INFO] = EVALUATESURFACERECURSION(...) replaces the nested sampling of
4: % Eq. (23) by a cached CONDITIONAL SMOOTHING SURFACE. It returns the same
5: % |X_0| x |S| object that innerNestedEstimate returns, so the two are
6: % interchangeable inside estimateNewFactor and differ only in how the inner
7: % integral is approximated.
8: %
9: % INDEXING CONVENTION. The specification is inconsistent about the index of
10: % R: section 7 writes R_1(x1,x2), indexing by the CONDITIONING variable,
11: % while section 9 writes R_r(xi_r,s) with R_H = a_H, indexing by LEVEL.
12: % This implementation follows section 9, which is the form the matrix
13: % recursion of section 10 actually implements, and maps the example onto it
14: % as:
15: %
16: %     xi_0 = x1,  xi_1 = l1,  H = 1
17: %     R_1(l1, x2) = a_1(l1, x2) = f(l1, x2)                (Eq. 38)
18: %     R_0(x1, x2) = int g_0(x1,l1) R_1(l1,x2) dl1         (Eq. 40, a_0 = 1)
19: %
20: % so this R_0 is exactly the surface written R_1(x1,x2) in section 7's
21: % Eq. (26), and Eq. (41) then reproduces Eq. (27). The fusion factor a_0 is
22: % identically 1 here because the only factor coupling l1 to the separator
23: % is already the terminal surface; putting it in both places would double
24: % count it.
25: %
26: % Finite implementation (Eq. 50):
27: %
28: %     R_0(a,rho) = sum_{b in N_0(a)} P_0(a,b) A_0(a,b,rho) R_1(b,rho)
29: %
30: % with P_0 the row-normalized transition matrix on the active successor set
31: % and A_0(a,b,rho) = Z_0(xi_0^(a)) * a_0(xi_1^(b), s^(rho)) = Z_0(xi_0^(a)).
32: %
33: % The gain claimed by the method comes from |N_r| << |X_1|, and from the
34: % surface being reusable across increments. Both are measured in INFO
35: % rather than asserted.
36: %
37: arguments
38:     af (1,1) core.ApproximateFactor
39:     g0 (1,1) core.Factor
40:     fuse (1,:) core.Factor
41:     omega (1,1) string
42:     sepVar (1,1) string
43:     S (1,:) double
44:     config (1,1) struct
45: end

```

```

46:
47: t0 = tic;
48:
49: if config.surfaceMode ~= "finite"
50:     error('methods:smoothed:modeNotImplemented', ...
51:         ['Surface mode "%s" is Mode B/C of specification section 11 and ' ...
52:         'arrives in iteration 4. Iteration 1 implements Mode A ("finite"), ' ...
53:         'which the spec requires to be validated first.'], config.surfaceMode);
54: end
55:
56: xi0 = af.Samples;           % X_0, |X_0| = B_0
57: B0 = numel(xi0);
58: Rs = numel(S);
59: omegaKey = matlab.lang.makeValidName(omega);
60: outerKey = matlab.lang.makeValidName(af.EliminatedVar);
61: sepKey = matlab.lang.makeValidName(sepVar);
62:
63: % --- Support X_1 of the inner path variable -----
64: % Built from the structural proposal itself (spec section 16.2: "samples
65: % inherited from the paper's structural sampling route"), pooled across the
66: % outer support so that one shared support serves every row. Pooling is what
67: % turns the nested sample TREE into a surface MATRIX.
68: [X1, supportInfo] = methods.smoothed.buildSamplingPath( ...
69:     af, g0, omega, config);
70: B1 = numel(X1);
71:
72: % --- Terminal surface R_1(b, rho) = a_1(xi_1^(b), s^(rho)) (Eq. 38) -----
73: R1 = ones(B1, 1);
74: for i = 1:numel(fuse)
75:     R1 = R1 .* fuse(i).evaluate(struct(omegaKey, X1(:, sepKey, S)));
76: end                                     % B1 x Rs
77:
78: % --- Transition weights P_0(a,b) on the active successor set (Eq. 46, 49) -
79: % q_0(xi_1 | xi_0) is proportional to g_0(xi_0, xi_1), and each support point
80: % carries the Voronoi width of its node so that the row sum approximates an
81: % integral rather than an equally-weighted average over a nonuniform support.
82: W = g0.evaluate(struct(outerKey, xi0(:, omegaKey, reshape(X1, 1, [])))); % B0 x B1
83: W = W .* reshape(supportInfo.nodeWeights, 1, []);
84:
85: [P0, active, activeInfo] = methods.smoothed.buildActiveSuccessors(W, config);
86:
87: % --- Fusion multiplier A_0(a,b, rho) = Z_0(xi_0^(a)) * a_0(...) -----
88: % a_0 == 1 for this path (see the indexing note above), so the tensor
89: % collapses to a per-row scalar and never needs to be materialized.
90: logZ0 = reshape(g0.logNormalizer(omega, struct(outerKey, xi0)), [], 1);
91: if isscalar(logZ0)
92:     logZ0 = repmat(logZ0, B0, 1);
93: end
94:
95: % --- Sparse recursion, Eq. (50) -----
96: % With a_0 constant in b, the row sum is a plain sparse matrix product.
97: %
98: % Note that Z_0 multiplies here and divides inside the row normalization of
99: % P_0, since sum_b g_0(a,b) w_b approximates Z_0(xi_0^(a)). The two cancel to
100: % leave sum_b g_0(a,b) R_1(b, rho) w_b, i.e. Eq. (26) evaluated by quadrature.
101: % Keeping both written out follows Eq. (47) literally and makes the estimator
102: % insensitive to an inaccurate Z_0.
103: R0 = exp(logZ0) .* (P0 * R1);           % B0 x Rs
104:
105: % --- Required numerical checks, specification section 16.3 -----
106: checks = methods.smoothed.surfaceChecks(R0, P0, active);
107:
108: info = struct( ...
109:     'estimator',      "RCS finite support (Eq. 50)", ...
110:     'mode',           config.surfaceMode, ...
111:     'numInnerSamples', 0, ...
112:     'numOuterSamples', B0, ...

```

```

113:     'supportX1',      B1, ...
114:     'separatorSupport', Rs, ...
115:     'activeSetSize',   activeInfo.K, ...
116:     'activeSetRule',   config.activeSetRule, ...
117:     'support',         X1, ...
118:     'supportInfo',     supportInfo, ...
119:     'R1',             R1, ...
120:     'transition',      activeInfo, ...
121:     'checks',          checks, ...
122:     'costModel',       "|X_0| |N_0| |S|", ...
123:     'predictedCost',   B0 * activeInfo.K * Rs, ...
124:     'storage',         B0 * Rs, ...
125:     'elapsed',         toc(t0));
126:
127: % The whole claim of the method is that the surface replaces nested
128: % sampling; recording zero inner samples makes that explicit and lets the
129: % cost-scaling test compare like with like.
130: end

```

buildSamplingPath.m: pooled support and quantile thinning

```

1: function [X1, info] = buildSamplingPath(af, g0, omega, config)
2: %BUILDSAMPLINGPATH Finite support X_r for the path variable (Eq. 42).
3: % [X1, INFO] = BUILDSAMPLINGPATH(AF, G0, OMEGA, CONFIG) builds the shared
4: % finite support of the inner path variable xi_1 = OMEGA.
5: %
6: % Specification section 16.2 says X_r should use "samples inherited from
7: % the paper's structural sampling route", so the support is drawn from the
8: % very same proposal q_0(xi_1 | xi_0) that the nested estimator uses. The
9: % difference is what happens next: the nested estimator keeps a separate
10: % private block of samples per outer point, forming a TREE of size
11: % |X_0| x M, while here the draws are POOLED into one support of size
12: % |X_1| that every row shares. That pooling is what makes a surface matrix
13: % possible, and it is also the method's exposure: a pooled support must
14: % still cover every region any single row cares about.
15: %
16: % The support is thinned to |X_1| = surfaceSupportSize by even quantile
17: % spacing rather than by taking the first draws, so that a pooled support
18: % built from a multimodal proposal retains all of its modes. Blind
19: % truncation here is the "support loss" failure mode of section 18.
20:
21: arguments
22:     af (1,1) core.ApproximateFactor
23:     g0 (1,1) core.Factor
24:     omega (1,1) string
25:     config (1,1) struct
26: end
27:
28: xi0 = af.Samples;
29: B0 = numel(xi0);
30: target = config.surfaceSupportSize;
31:
32: % Draw enough structural samples that the pooled support covers the union of
33: % the per-row proposals, then thin.
34: perRow = max(2, ceil(target / max(B0, 1)) + 1);
35: given = struct(matlab.lang.makeValidName(af.EliminatedVar), xi0);
36: pool = reshape(g0.sample(omega, given, perRow), [], 1);
37:
38: pool = sort(pool);
39: raw = numel(pool);
40:
41: if raw <= target
42:     X1 = pool;
43: else
44:     % Even quantile spacing preserves the shape of the pooled distribution,
45:     % including separated modes.
46:     q = linspace(0, 1, target);

```

```

47:     X1 = quantile(pool, q);
48:     X1 = unique(X1(:), 'stable');
49: end
50:
51: X1 = reshape(X1, [], 1);
52:
53: % Node weights. The support points are NOT equally spaced -- they are
54: % quantiles of a pooled proposal, so they crowd where the proposal is dense.
55: % Row-normalizing raw g_0 values over such a support would silently weight
56: % each point equally, which is a quadrature against the wrong reference
57: % measure and biases the surface toward the crowded region. Attaching the
58: % Voronoi width of each node turns the recursion into a genuine nonuniform
59: % quadrature and makes it independent of how the support was built, so a
60: % plain grid (also permitted by section 16.2) works unchanged.
61: nodeWeights = localNodeWeights(X1);
62:
63: info = struct( ...
64:     'rule',          "structural proposal, pooled and quantile-thinned", ...
65:     'poolSize',      raw, ...
66:     'perRowDraws',   perRow, ...
67:     'cardinality',   numel(X1), ...
68:     'range',         [min(X1) max(X1)], ...
69:     'requested',     target, ...
70:     'nodeWeights',   nodeWeights, ...
71:     'weightRule',    "Voronoi width of each support point");
72: end
73:
74: % -----
75: function w = localNodeWeights(x)
76: %LOCALNODEWEIGHTS Voronoi (midpoint) widths of a sorted 1-D support.
77: x = x(:);
78: n = numel(x);
79: if n == 1
80:     w = 1;
81:     return
82: end
83: edges = [x(1); 0.5 * (x(1:end-1) + x(2:end)); x(end)];
84: w = diff(edges);
85:
86: % A degenerate support (repeated points) would give zero-width nodes; fall
87: % back to uniform weights rather than producing an all-zero transition row.
88: if all(w == 0)
89:     w = ones(n, 1) / n;
90: end
91: end

```

buildActiveSuccessors.m: active successor truncation and renormalization

```

1: function [P, active, info] = buildActiveSuccessors(W, config)
2: %BUILDACTIVESUCCESSORS Active successor sets N_r(a) and weights P_r (Eq. 49).
3: % [P, ACTIVE, INFO] = BUILDACTIVESUCCESSORS(W, CONFIG) turns the dense
4: % unnormalized transition weights W (|X_r| x |X_{r+1}|) into a sparse
5: % row-stochastic matrix supported on K = |N_r(a)| successors per row.
6: %
7: % Selection rules, all listed in specification section 11.1:
8: % "transition" keep the K highest transition weights (default)
9: % "nearest" keep the K nearest successors by weight rank from the
10: % row maximum, i.e. a contiguous neighbourhood
11: % "random" keep K uniformly at random, as a control
12: %
13: % Rows are renormalized over their active set, which section 11.1 requires
14: % explicitly: without it the sparsification silently loses mass and the
15: % surface is biased low rather than merely coarse.
16: %
17: % K = Inf selects the dense update of Eq. (48). Comparing dense against
18: % sparse on the same support is how the app separates "the recursion is
19: % wrong" from "the active set is too small".

```



```

20:
21: arguments
22:     W (:,:) double
23:     config (1,1) struct
24: end
25:
26: [B0, B1] = size(W);
27: K = config.activeSetSize;
28:
29: if isinf(K) || K >= B1
30:     % Dense update, Eq. (48).
31:     rowSum = sum(W, 2);
32:     P = W ./ max(rowSum, realmin);
33:     P(rowSum <= 0, :) = 0;
34:     active = repmat((1:B1).'), B0, 1);
35:     info = struct('K', B1, 'rule', "dense (Eq. 48)", 'sparse', false, ...
36:         'meanRetainedMass', 1, 'minRetainedMass', 1, 'density', 1);
37:     return
38: end
39:
40: K = round(K);
41: active = cell(B0, 1);
42: rows = zeros(B0 * K, 1);
43: cols = zeros(B0 * K, 1);
44: vals = zeros(B0 * K, 1);
45:
46: retained = zeros(B0, 1);
47: totalMass = sum(W, 2);
48:
49: for a = 1:B0
50:     w = W(a, :);
51:     switch config.activeSetRule
52:         case "transition"
53:             [~, ord] = sort(w, 'descend');
54:             idx = ord(1:K);
55:         case "nearest"
56:             [~, peak] = max(w);
57:             half = floor(K / 2);
58:             lo = max(1, min(peak - half, B1 - K + 1));
59:             idx = lo:(lo + K - 1);
60:         case "random"
61:             idx = randperm(B1, K);
62:     end
63:     idx = sort(idx(:));
64:
65:     wk = w(idx);
66:     s = sum(wk);
67:     if totalMass(a) > 0
68:         retained(a) = s / totalMass(a);
69:     else
70:         retained(a) = 0;
71:     end
72:
73:     if s > 0
74:         wk = wk / s;           % renormalize over the active set
75:     else
76:         wk = zeros(size(wk));
77:     end
78:
79:     span = (a-1)*K + (1:K);
80:     rows(span) = a;
81:     cols(span) = idx;
82:     vals(span) = wk;
83:     active{a} = idx;
84: end
85:
86: P = sparse(rows, cols, vals, B0, B1);

```

```

87:
88: info = struct( ...
89:     'K',          K, ...
90:     'rule',       config.activeSetRule, ...
91:     'sparse',     true, ...
92:     'meanRetainedMass', mean(retained), ...
93:     'minRetainedMass', min(retained), ...
94:     'retainedMass', retained, ...
95:     'density',    K / B1);
96: end

```

6. NF-iSAM: Conflicts, Limitations, and Required Fixes

Overall status: the code is conceptually aligned with NF-iSAM because it builds Bayes-tree clique conditional samplers and samples top-down. It is not an exact reproduction of the paper implementation. The main deviations are tree rebuilding/reuse logic, sample-based separator propagation, a custom MATLAB flow trainer, and a reduced training cap.

Issue	Location	Why this conflicts / limits paper fidelity	What the AI should change
Bayes tree rebuilt; reuse by identity	incrementalUpdate.m	The paper emphasizes incremental Bayes-tree updates and retraining affected cliques/subtree. Current code rebuilds the tree and reuses if subtree/clique identity matches.	Add result.deviations.rebuiltTree=true. Do not call it surgical NF-iSAM update. Keep cache-hit diagnostics.
Child separator propagation by samples/resampling	combineChildSeparators.m	The ideal method propagates learned separator density information. Current code combines child separators through samples and resampling; partial overlap can lose learned correlations.	Promote warnings from combineChildSeparators into result.logs and Diagnostics tab. Consider density-object propagation later.
Custom MATLAB flow training	conditionalSamplerTrainer.m and flows/trainFlow.m	Paper implementation uses PyTorch and specific RQ-spline setup. Current code uses custom MATLAB flow class, Adam settings, and runtime choices.	Label as MATLAB-native approximation. Log architecture and training args per clique.
Training cap reduced to 200 iterations	commonMethodConfig.m:L118-L135	Paper stopping rule can allow up to larger iteration count; current cap is an app-runtime compromise.	Report stoppedBy for each clique. Add warning when stoppedBy=max_iter. Provide optional paperLikeTraining profile.
NF-iSAM curves are KDE from samples	runNFISAMMethod.m posterior packaging	NF-iSAM does not produce f_new or unnormalized generated separator factors. Grid curves are visualization reconstructions from samples.	Keep massError=NaN for NF-iSAM. Figure captions must say KDE-from-samples.
Factor-evaluation count is not comparable alone	runNFISAMMethod.m metrics	NF-iSAM can report zero factor evaluations while spending cost on flow fits. Comparing factor evals alone unfairly favors NF-iSAM.	Report trainTime, numTrainedFlows, flowIterations, and trainSamples alongside factor evaluations.

Relevant current code excerpt:

runNFISAMMethod.m: NF-iSAM wrapper, batch/replay dispatch, posterior sampling

```

1: function result = runNFISAMMethod(caseData, config)
2: %RUNNFISAMMETHOD NF-iSAM: Bayes tree of normalizing flows.
3: %   RESULT = RUNNFISAMMETHOD(CASEDATA, CONFIG) trains one conditional flow per
4: %   Bayes tree clique (Algorithms N1-N3), draws the joint posterior root to
5: %   leaf, and returns the unified result structure of app specification
6: %   section 8.
7: %
8: %   THE WHOLE METHOD, in the two calls this function makes:
9: %
10: %       METHODS.NFISAM.INCREMENTALUPDATE   the graph becomes a Bayes tree and
11: %                                           every clique gets a flow fitted to
12: %                                           simulated samples, leaf to root
13: %       METHODS.NFISAM.SAMPLEJOINTPOSTERIOR the flows are read root to leaf,
14: %                                           each clique conditioned on the
15: %                                           separator values its parent drew
16: %
17: %   Everything else here is the contract: shaping what those two return into
18: %   the fields the comparative layer, the map view and the metrics table
19: %   already read from the other two methods.
20: %
21: %   THE BUILD ORDER of NF-iSAM spec section 15, and where it stands:
22: %       1. factor graph -> Bayes tree conversion and update           core.bayesTree
23: %       2. Clique objects with F_C, S_C, z_C                         core.BayesTreeClique
24: %       3. Algorithm N1 TrainingSampleSimulator                     methods.nfisam
25: %       4. autoregressive RQ spline flow, density and sampling       flows.RQSplineFlow

```

Static source-level handoff; verify with MATLAB tests after edits.

Smoothed Slices Project — Implementation Conflict and Correction Brief

```

26: %      5. Algorithm N2 ConditionalSamplerTrainer: train      methods.nfisam
27: %      T tilde on ordered samples (O, S, F), fix O = o,      ConditionalSampler
28: %      partition into T_S and T_F
29: %      6. Algorithm N3 incrementalUpdate: update the graph,      methods.nfisam
30: %      rebuild the tree, retrain leaf-to-root where the      cliqueKeys
31: %      clique identity changed, reuse the rest
32: %      7. root-to-leaf joint posterior sampling      methods.nfisam
33: %      sampleJointPosterior
34: %
35: %      TWO MODES, matching METHODS.RUNGENERALCORE so that a comparative run puts
36: %      all three methods in the same regime. By default the tree is built once
37: %      with all the data present and every clique is trained from scratch, which
38: %      is the batch pass. With CONFIG.INCREMENTAL the mission is replayed
39: %      k = 1..K and each increment reuses the flows whose cliques did not change;
40: %      RESULT.INCREMENT then carries the reuse fraction per increment, which is
41: %      the number the incremental claim lives or dies by. In batch there is
42: %      nothing to reuse and reporting a reuse rate would be reporting a zero as
43: %      if it were a result.
44: %
45: %      WHAT THIS METHOD RETURNS THAT THE OTHER TWO DO NOT, and the reverse.
46: %      Slices and Smoothed Slices carry a finite separator support and a
47: %      generated factor evaluated on it; NF-iSAM has neither, because a flow is
48: %      a density with no support to speak of. What it has instead is a normalized
49: %      posterior it can sample from directly, so RESULT.POSTERIOR.ESTIMATOR
50: %      declares itself NORMALIZED and METHODS.SCOREAGAINSTREFERENCE compares it
51: %      against the reference's normalized marginal rather than against the
52: %      unnormalized f_new. The mass of Eq. (19)'s generated factor is not among
53: %      this method's outputs and is reported as NaN rather than guessed at.
54: %
55: %      THE MARGINAL CURVE IS A RECONSTRUCTION, and is the one place this method
56: %      is scored through something it did not produce. NF-iSAM's answer is a set
57: %      of joint draws; the quadrature reference is a curve on a grid, and the
58: %      two only meet if one is turned into the other. The draws are turned into
59: %      a density by a Gaussian kernel estimate, marked as such on
60: %      RESULT.POSTERIOR.ESTIMATOR.RECONSTRUCTION, and used for nothing else: the
61: %      MMD and the RMSE that the comparison actually turns on are computed from
62: %      the draws themselves, where no bandwidth choice can reach them.
63:
64: arguments
65:     caseData (1,1) struct
66:     config (1,1) struct = methods.commonMethodConfig()
67: end
68:
69: seedInfo = utils.setRandomSeed(config.seed);
70: if isfield(caseData, 'counter') && ~isempty(caseData.counter)
71:     caseData.counter.reset();
72: end
73:
74: incremental = isfield(config, 'incremental') && config.incremental ...
75:     && isfield(caseData, 'increments') && ~isempty(caseData.increments);
76:
77: % The split is lopsided in a way the other two methods are not: fitting the
78: % flows is essentially all of the cost and drawing from them is essentially
79: % none of it, so the bar gives the training pass almost everything.
80: p = utils.progressOf(config);
81:
82: % --- Forward: the tree and its flows -----
83: tf = tic;
84: if incremental
85:     [state, updates] = localReplay(caseData, config, p.sub(0, 0.95));
86: else
87:     [state, updates] = localBatch(caseData, config, p.sub(0, 0.95));
88: end
89: runtimeForward = toc(tf);
90: forwardEvals = localEvals(caseData);
91:
92: % --- Backward: the root-to-leaf draw -----

```

Static source-level handoff; verify with MATLAB tests after edits.

Smoothed Slices Project — Implementation Conflict and Correction Brief

```

93: p.report(0.95, "drawing the joint posterior root to leaf");
94: tb = tic;
95: [draws, drawInfo] = methods.nfisam.sampleJointPosterior(state, config.numBackwardSamples);
96: runtimeBackward = toc(tb);
97: backwardEvals = localEvals(caseData) - forwardEvals;
98:
99: % --- The contract -----
100: result = struct();
101: result.methodName = "NF-iSAM";
102: result.status      = "ok";
103: result.config      = utils.serializableConfig(config);
104: result.case        = struct('name', caseData.name, 'variant', caseData.variant, ...
105:                             'displayName', caseData.displayName, ...
106:                             'settings', caseData.settings);
107: result.seed        = seedInfo;
108:
109: result.increment = localIncrementRecords(updates, caseData, incremental);
110:
111: result.posterior = localPosterior(draws, state, caseData, config);
112:
113: result.bayesNet = localBayesNet(state.tree);
114:
115: health = localHealth(updates, state, drawInfo);
116:
117: result.metrics = struct( ...
118:     'runtimeTotal',    runtimeForward + runtimeBackward, ...
119:     'runtimeForward',  runtimeForward, ...
120:     'runtimeBackward', runtimeBackward, ...

```

incrementalUpdate.m: graph copy, order handling, tree rebuild, cache/reuse loop

```

1: function [state, info] = incrementalUpdate(state, newFactors, opts)
2: %INCREMENTALUPDATE Algorithm N3 of the NF-iSAM spec.
3: % [STATE, INFO] = INCREMENTALUPDATE(STATE, NEWFACTORS, ...) takes the state
4: % returned by METHODS.NFISAM.INITIALSTATE or by a previous update, adds the
5: % new measurements, and returns a state in which every clique of the Bayes
6: % tree has a trained conditional sampler.
7: %
8: % The three steps of the algorithm, and where each one is:
9: %
10: % 1. G.update(f, Theta)                the graph copy below
11: % 2. T_delta <- T.extract(f, Theta)    CORE.AFFECTEDSUBTREE
12: % 3. leaf-to-root over T_delta:        the traversal below
13: %     x, o <- TrainingSampleSimulator(C) METHODS.NFISAM.TRAININGSAMPLESIMULATOR
14: %     p(F|S), p(S) <- ConditionalSamplerTrainer(x, o)
15: %     append p(S_C) to the parent      METHODS.NFISAM.COMBINECHILDSEPARATORS
16: %
17: % WHAT IS RETRAINED IS NOT WHAT STEP 2 RETURNS, and the difference is the
18: % one substantive departure here. Step 2 names the cliques the new factors
19: % change. This implementation rebuilds the whole Bayes tree after the graph
20: % update rather than re-eliminating it surgically, and a rebuild can reshape
21: % a clique BELOW the changed subtree: adding a loop closure can split a
22: % clique whose own variables nobody measured, because its merge into its
23: % parent depended on the parent's separator. Retraining is therefore driven
24: % by clique identity -- METHODS.NFISAM.CLIQUEKEYS -- and step 2's answer is
25: % computed alongside and reported. INFO.outsideSubtree is the cliques where
26: % they disagreed; on a plain append it is empty, and a run in which it never
27: % empties is a sign the elimination order is churning.
28: %
29: % REUSE IS THE POINT. A clique outside the retraining set keeps the flow it
30: % already has, at the cost of nothing: no simulation, no fit, no iteration.
31: % INFO.reuseFraction is the direct counterpart of the StepCache hit rate on
32: % the Slices side, and the one number that says whether the incremental
33: % claim held on this case.
34: %
35: % THE ELIMINATION ORDER IS APPENDED TO, not recomputed, for the reason
36: % METHODS.GENERAL.RUNINCREMENTALGENERAL gives -- and it matters more here.

```

Smoothed Slices Project — Implementation Conflict and Correction Brief

```

37: % There, a churned order costs a cache of evaluated surfaces. Here it
38: % rennumbers and reshapes cliques, so every subtree key changes and every
39: % flow in the tree is fitted again from scratch. The order is only
40: % recomputed when appending would break Lemma 1, and INFO.orderSource says
41: % which happened.
42: %
43: % Options
44: %     NewVariables      CORE.VARIABLE array for names the new factors
45: %                       introduce. Not inferred: a variable carries a domain,
46: %                       and inventing one would put a made-up support into the
47: %                       graph the other two methods also read.
48: %     NumSamples        n_train per clique; spec section 9 default is 2000,
49: %                       which FLOWS.TRAINFLOW documents as measurably thin.
50: %     Training          forwarded to FLOWS.TRAINFLOW.
51: %     AngleColumns      variable name -> which of its columns are orientations,
52: %                       for the whole graph. Each clique is given the part of
53: %                       it that names its own variables, because
54: %                       METHODS.NFISAM.CONDITIONALSAMPLERTRAINER refuses a
55: %                       declaration it cannot apply -- rightly, since silently
56: %                       ignoring it there would leave orientations unwrapped.
57: %     Reuse             false retrains every clique. The counterfactual the
58: %                       incremental claim is measured against.
59: %     Seed              seeds the global stream once, so an update is
60: %                       reproducible end to end: simulation, training and the
61: %                       resampling in COMBINECHILDSEPARATORS all draw from it.
62: %
63: arguments
64:     state (1,1) struct
65:     newFactors (1,:) core.Factor = core.Factor.empty(1,0)
66:     opts.NewVariables (1,:) core.Variable = core.Variable.empty(1,0)
67:     opts.NumSamples (1,1) double {mustBeInteger, mustBePositive} = 2000
68:     opts.Training (1,1) struct = struct()
69:     opts.AngleColumns (1,1) struct = struct()
70:     opts.MultimodalMeasurement (1,1) string = "maxWeight"
71:     opts.Reuse (1,1) logical = true
72:     opts.Seed = []
73:     %PROGRESS A utils.ProgressReporter, or [] for a run nobody is watching.
74:     % An option rather than a config field because this function takes no
75:     % config: it is called with the pieces of one, and adding the whole
76:     % struct just to carry a reporter would blur what it actually depends on.
77:     opts.Progress = []
78: end
79:
80: tStart = tic;
81: if ~isempty(opts.Seed)
82:     rng(opts.Seed);
83: end
84:
85: % --- Step 1: G.update(f, Theta) -----
86: % On a copy: the state is a value struct held across increments, and
87: % CORE.FACTORGRAPH is a handle, so appending in place would reach back into
88: % every earlier state the caller kept and into the case graph itself.
89: graph = state.graph.copy();
90:
91: for i = 1:numel(opts.NewVariables)
92:     v = opts.NewVariables(i);
93:     if any(graph.VariableNames == v.Name)
94:         error('methods:nfisam:incrementalUpdate:duplicateVariable', ...
95:             '%s is already in the graph.', v.Name);
96:     end
97:     graph.addVariable(v);
98: end
99:
100: for i = 1:numel(newFactors)
101:     unknown = newFactors(i).Scope(~ismember(newFactors(i).Scope, graph.VariableNames));
102:     if ~isempty(unknown)
103:         error('methods:nfisam:incrementalUpdate:unknownVariable', ...

```

Static source-level handoff; verify with MATLAB tests after edits.

Smoothed Slices Project — Implementation Conflict and Correction Brief

```

104:         ['Factor %s names %s, which the graph does not have. Pass it as ' ...
105:         'NewVariables: a variable carries a domain, and this function ' ...
106:         'will not invent one.'], newFactors(i).Name, strjoin(unknown, ', '));
107:     end
108:     graph.addFactor(newFactors(i));
109: end
110:
111: % --- The elimination order: keep the prefix, append the new -----
112: present = graph.VariableNames;
113: kept    = state.order(ismember(state.order, present));
114: appended = present(~ismember(present, kept));
115: order    = [kept appended];
116: orderSource = "appended";
117: try
118:     graph.validateEliminationOrder(order);
119: catch
120:     order = core.eliminationOrder(graph);
121:     orderSource = "recomputed";
122: end
123:
124: % --- Step 2: T_delta <- T.extract(f, Theta) -----
125: tree = core.bayesTree(graph, order);
126: keys = methods.nfisam.cliqueKeys(tree);
127: numC = numel(tree.cliques);
128:
129: changed = string.empty(1, 0);
130: for i = 1:numel(newFactors)
131:     changed = [changed newFactors(i).Scope]; %#ok<AGROW>
132: end
133: changed = unique(changed, 'stable');
134:
135: if isempty(changed) || state.numUpdates == 0
136:     % Nothing to extract from: the first update has no previous tree to
137:     % compare against, so the changed subtree is the whole tree.
138:     subtree = keys.leafToRoot;
139: else
140:     subtree = core.affectedSubtree(tree, changed);
141: end
142:
143: % --- Step 3: leaf to root -----
144: % Every clique is visited, not only those of T_delta, because the retraining
145: % set is decided by key and a clique with no cache entry has no sampler at
146: % all. The set is upward closed -- a clique's subtree key contains its
147: % children's -- so deepest-first is still the order that has each clique's
148: % children ready before it is trained.
149: samplers = cell(1, numC);
150: recCell   = cell(1, numC);
151: trained   = zeros(1, 0);
152: reused    = zeros(1, 0);
153:
154: % One clique is the natural boundary: a flow fit is seconds to tens of
155: % seconds and is the unit of work Algorithm N3 either does or reuses. There
156: % is no finer place to stop -- flows.trainFlow owns the iteration loop, and
157: % stopping inside one would leave a half-fitted flow in the cache under a
158: % subtree key that claims it is finished.
159: p = utils.progressOf(struct('progress', opts.Progress));
160: done = 0;

```

combineChildSeparators.m: sample-based child separator combination and overlap warning

```

1: function [given, info] = combineChildSeparators(samplers, n)
2: %COMBINECHILDSEPARATORS Algorithm N3's "append p(S_C) to the parent clique".
3: % [GIVEN, INFO] = COMBINECHILDSEPARATORS(SAMPLERS, N) draws N samples from
4: % each child's separator density and returns them as the GIVEN dictionary
5: % that METHODS.NFISAM.TRAININGSAMPLESIMULATOR conditions on.
6: %
7: % WHY SAMPLES AND NOT A CORE.FACTOR. Spec section 7 says to append p(S_C) to

```

Smoothed Slices Project — Implementation Conflict and Correction Brief

```

8: % the parent as a factor, and Algorithm N1 does not evaluate factors -- it
9: % simulates from them. So what the parent needs from a separator density is
10: % draws, which is exactly what the appended factor would have been asked for.
11: % Passing it as a CORE.FACTOR instead would be worse in two ways that matter:
12: % a separator wider than two variables is refused by METHODS.NFISAM.FACTORROLE,
13: % and Algorithm N1 samples a factor one variable at a time, which would throw
14: % away the correlation between separator variables -- the one thing the child
15: % learned that the parent cannot rederive.
16: %
17: % TWO CHILDREN THAT SHARE SEPARATOR VARIABLES are the only hard case, and
18: % there are three of them:
19: %
20: % disjoint the separators do not meet. The two densities are separate
21: % factors over separate variables, so drawing from each
22: % independently is exact.
23: % contained a later child's separator is entirely among variables already
24: % drawn. Its density is then a second factor over the same
25: % variables, and the product is formed by weighting the
26: % existing draws by that density and resampling. Exact in the
27: % Monte Carlo limit, and the effective sample size is reported
28: % so that a product too sharp for N draws is visible rather
29: % than merely quiet.
30: % partial a later child's separator meets the drawn variables without
31: % being contained in them. The product would need the child's
32: % density marginalized onto the overlap, which a flow does not
33: % give in closed form. The overlap keeps the earlier child's
34: % draws and the rest is taken from this one; the approximation
35: % is warned about, named, and recorded in INFO.
36: %
37: % RESAMPLING APPLIES TO EVERY VARIABLE DRAWN SO FAR, not only the ones being
38: % weighted. The draws are one joint sample set; reindexing part of it would
39: % break exactly the correlation this function exists to carry upward.
40:
41: arguments
42:     samplers (1,:) cell
43:     n (1,1) double {mustBeInteger, mustBePositive}
44: end
45:
46: given = struct();
47: names = string.empty(1, 0);
48:
49: info = struct('names', string.empty(1,0), 'numChildren', numel(samplers), ...
50:             'combined', string.empty(1,0), 'effectiveSampleSize', [], ...
51:             'approximated', string.empty(1,0));
52:
53: for k = 1:numel(samplers)
54:     s = samplers{k};
55:     if ~isa(s, 'methods.nfisam.ConditionalSampler')
56:         error('methods.nfisam.combineChildSeparators:notASampler', ...
57:             ['Child %d is a %s. Algorithm N3 trains leaf to root, so every ' ...
58:             'child already has its sampler by the time the parent asks.'], ...
59:             k, class(s));
60:     end
61:
62:     sep = s.Separator;
63:     if isempty(sep)
64:         continue % a child with no separator shares nothing with this parent
65:     end
66:
67:     known = ismember(sep, names);
68:
69:     if ~any(known)
70:         % --- disjoint: an independent factor over new variables -----
71:         D = methods.nfisam.unpackSamples(s.sampleSeparator(n), sep, s.separatorWidths());
72:         for v = sep
73:             key = matlab.lang.makeValidName(v);
74:             given.(key) = D.(key);

```

Static source-level handoff; verify with MATLAB tests after edits.

```

75:         end
76:         names = [names sep]; %#ok<AGROW>
77:
78:     elseif all(known)
79:         % --- contained: form the product by weighting and resampling -----
80:         X = methods.nfisam.packSamples(given, sep);
81:         lw = s.separatorLogProb(X);
82:         w = exp(lw - max(lw));
83:         w = w / sum(w);
84:
85:         info.combined(end+1) = strjoin(sep, ",");
86:         info.effectiveSampleSize(end+1) = 1 / sum(w.^2);
87:
88:         idx = localSystematicResample(w);
89:         for f = string(fieldnames(given)).'
90:             given.(f) = given.(f)(idx, :);
91:         end
92:
93:     else
94:         % --- partial: the overlap cannot be multiplied, so say so -----
95:         shared = sep(known);
96:         warning('methods:nfisam:combineChildSeparators:partialOverlap', ...
97:             ['Two children of this clique share %s but not their whole ' ...
98:             'separators, so their densities cannot be multiplied by ' ...
99:             'reweighting. The shared variables keep the first child''s ' ...
100:            'draws and the correlation the second learned across %s is ' ...
101:            'lost.'], strjoin(shared, ', '), strjoin(sep, ', '));
102:         info.approximated(end+1) = strjoin(shared, ",");
103:
104:         D = methods.nfisam.unpackSamples(s.sampleSeparator(n), sep, s.separatorWidths());
105:         for v = sep(~known)
106:             key = matlab.lang.makeValidName(v);
107:             given.(key) = D.(key);
108:         end
109:         names = [names sep(~known)]; %#ok<AGROW>
110:     end
111: end
112:
113: info.names = names;
114: end
115:
116: % -----
117: function idx = localSystematicResample(w)
118: %LOCALSYSTEMATICRESAMPLE Indices drawn in proportion to W, one stratum each.
119: % Systematic rather than multinomial: it draws one uniform instead of N and
120: % has strictly lower variance, which is the difference between a product
121: % density that survives being formed and one that collapses onto a handful
122: % of distinct rows.
123: n = numel(w);
124: edges = [0; cumsum(w(:))];
125: edges(end) = 1; % against rounding, so DISCRETIZE covers 1
126: u = ((0:n-1)' + rand) / n;
127: idx = discretize(u, edges);
128: idx(isnan(idx)) = n; % u == 1 exactly
129: end

```

commonMethodConfig.m: NF-iSAM training sample count and reduced iteration cap

```

118: % --- NF-iSAM (spec section 9) -----
119: % n_train, the samples Algorithm N1 simulates per clique. The authors'
120: % value, and flows.trainFlow documents where it is measurably thin: a 4-D
121: % clique fitted from 2000 samples overfits badly enough to shrink a
122: % unit-variance marginal, with a loss curve that flattens as if converged.
123: opts.nfisamTrainSamples(1,1) double {mustBeInteger, mustBePositive} = 2000
124: % Forwarded verbatim to flows.trainFlow. MaxIterations is capped well
125: % below that function's own ceiling of 2000: a comparative run trains one
126: % flow per clique and the grid world has six, so the full ceiling makes a

```



```

127: % single Run All take tens of minutes. 200 leaves the authors' stopping
128: % rule room to fire -- it cannot trigger before iteration 100 and needs a
129: % 50-iteration window on each side of that -- while keeping a clique to
130: % about twenty seconds. Measured on the two-pose range benchmark at
131: % n_train = 2000, the relative L1 against quadrature is 0.035 at 100
132: % iterations, 0.050 at 150 and 0.037 at 250: flat inside its own Monte
133: % Carlo noise, while the runtime is not. The cap is an implementation
134: % choice and INFO.stoppedBy on every fit says whether it bound.
135: opts.nfisamTraining (1,1) struct = struct('MaxIterations', 200)
136: % false retrains every clique on every increment: the counterfactual the

```

7. Exact Code-Touch Instructions for the Next AI

The following modifications are recommended before any deeper algorithmic rewrite. They are intentionally minimal and mostly add clarity, diagnostics, and paper-fidelity flags.

1. **Add a method-fidelity/deviation field to every result:** Add result.implementation or result.deviations with fields such as methodFamily, paperFidelity, deviations, approximationLevel, engine, and warnings. Populate it in runInferenceCore, runGeneralCore, and runNFISAMMethod.
2. **Correct Slices wording:** Replace “reproduced faithfully” in runSlicesMethod.m with a qualified statement: paper-faithful for the controlled Slices demonstrator; documented approximation in general-engine mode.
3. **Separate paper-literal and generalized engines in UI/report:** Where the app displays method cards or diagnostics, print engine = demo_slices or general_importance_sampling. Make this visible in exported diagnostics.md and figures.
4. **Surface Smoothed Slices scope:** In runSmoothedSlicesMethod.m and methodCard.m, state finite-support Mode A, one-level Eq.16 path, and active-set general mode. Avoid claiming basis/regression surfaces are implemented.
5. **Improve active-set safety:** In buildActiveSuccessors.m/evaluateSurfaceRecursion.m, compute retainedMassMin, retainedMassMean, retainedMassMedian, and warn when retainedMassMin is low.
6. **Improve NF-iSAM diagnostics:** In incrementalUpdate.m and runNFISAMMethod.m, export treeWasRebuilt, reuseRule, numTrained, numReused, flowIterations, stoppedBy, and childSeparatorWarnings.
7. **Create paperLike and appFast config presets:** Do not delete the current fast settings. Add a preset that uses a more paper-like training budget where feasible, and show which preset was used.
8. **Add acceptance tests:** Add small deterministic tests for Eq.16: Slices nested vs quadrature; RCS dense support vs quadrature; sparse RCS retained mass behavior; general engine exactGeneratedFactors small-case check; NF-iSAM smoke test with flow convergence diagnostics.

8. Recommended Result Metadata Contract

Add this structure or an equivalent one to every method result. This gives the AI/developer a precise target and prevents future overclaiming.

Proposed metadata block

```

result.implementation = struct( ...
    'methodFamily',      "slices" | "smoothed_slices" | "nfisam", ...
    'paperReference',    "Slices Eq. (11)-(16)" | "NF-iSAM Algorithms N1-N3" | "proposed extension",
    ...
    'paperFidelity',     "paper_demo" | "implementation_generalization" | "prototype_extension", ...
    'engine',            "three_node_demo" | "general_importance_sampling" | "bayes_tree_flow", ...
    'mainDeviation',     string(...), ...
    'deviationList',     string(...), ...
    'scopeWarning',     string(...), ...
    'canClaimPaperExact', false);

```

9. Method-Specific Modification Checklist

9.1 Slices checklist

- Update runSlicesMethod.m comments to remove unqualified “reproduced faithfully.”
- In estimateNewFactor.m, when scalar separator restriction is triggered, return a clear diagnostic when possible instead of only erroring deep in the call stack.
- In runInferenceCore.m, set result.implementation.paperFidelity = “paper_demo” only when using the three-node/non-general engine.
- In runGeneralCore.m or forwardEliminationGeneral.m, set result.implementation.paperFidelity = “implementation_generalization”.
- Expose generated-factor lookup diagnostics in diagnosticsReport.m and exportAllFiguresAndDiagnostics.m.
- Add a small Eq.16 regression test against referenceTwoPoseQuadrature.

9.2 Smoothed Slices checklist

- Keep provenance.notInPaper=true and show it in UI/report/exports.
- In evaluateSurfaceRecursion.m, state in info.estimator that this is finite-support, one-level RCS unless full recursive path support is added.
- In buildSamplingPath.m, store support construction metadata: pooledSamples, quantileThinning, nodeWeights, supportSize.
- In buildActiveSuccessors.m, export retained-mass statistics and warn when sparse truncation is severe.
- In forwardEliminationGeneral.m, rename current general-engine behavior if it is only active-set sparse Slices, or implement a true R_r recursion there.
- Do not expose surfaceMode=“basis” or “regression” as usable choices until implemented.

9.3 NF-iSAM checklist

- Add explicit “tree rebuilt, reuse by clique identity” deviation to each result.
- Promote combineChildSeparators warnings into result.logs and the Diagnostics tab.
- Log per-clique training status: num samples, iterations, stoppedBy, final loss, holdout loss if available.
- Add paperLikeTraining preset with larger MaxIterations; keep current appFast preset for interactive use.
- Ensure all NF-iSAM grid curves are labeled KDE from posterior draws, not method-generated f_new.
- Keep mass error as NaN or not applicable for NF-iSAM.

10. Acceptance Tests the AI Should Add or Run

Test	Setting	Expected evidence
Eq.16 Slices vs quadrature	Two-pose range Gaussian	Relative L1 / mass error below chosen tolerance; generated factor shape matches quadrature.
Eq.16 RCS dense support vs quadrature	Smoothed Slices with high support, dense active set	RCS approximates the same inner integral as nested Slices; report R surface error if reference available.
Sparse RCS retained mass	Vary activeSetSize K	Accuracy should degrade smoothly; warning when retainedMassMin is low.
General engine identity check	activeSetSize=Inf in current general path	If code comment claims identical behavior with Slices, assert results are identical or change comment.
NF-iSAM smoke test	Small Gaussian case	Samples have plausible mean/covariance; all clique flow trainings report stoppedBy and iteration counts.
Incremental reuse sanity	Small multi-increment case	Cache hits/reuse are logged; early-stopping warnings appear if sample rows are reused.

11. Phrases the AI Must Avoid

- Avoid: “Slices is faithfully reproduced” without specifying the controlled demonstrator engine.
- Avoid: “Smoothed Slices is from the Slices paper.” Correct: “Smoothed Slices is a proposed extension based on the Slices factorization.”
- Avoid: “RCS removes the integral.” Correct: “RCS reorganizes/approximates the inner integral as a reusable conditional-smoothing surface.”

- Avoid: “Sparse RCS is exact.” Correct: “Sparse RCS is a truncated/renormalized conditional expectation unless retained mass is effectively one.”
- Avoid: “NF-iSAM factor evaluations are zero, therefore it is free.” Correct: “NF-iSAM shifts cost from factor evaluation to training normalizing-flow clique samplers.”
- Avoid: “NF-iSAM produces f_{new} .” Correct: “NF-iSAM produces normalized posterior samples; grid curves are KDE reconstructions for visualization.”

12. Appendix: Source File Map

File	Why it matters
runSlicesMethod.m	Slices wrapper; sets innerEstimator="nested"; comments need qualification.
runSmoothedSlicesMethod.m	Smoothed wrapper; sets innerEstimator="rcs"; records proposed-extension provenance; chooses activeSetSize for general engine.
runInferenceCore.m	Shared three-node driver for Slices/Smoothed; dispatches to general engine when caseData.engine=="general".
+slices/estimateNewFactor.m	Core split point between nested and RCS; scalar separator restriction in Lemma-1 path.
+slices/innerNestedEstimate.m	Paper-style nested estimator for $R(x_i, s)$.
+smoothed/evaluateSurfaceRecursion.m	Finite-support one-level RCS implementation.
+smoothed/buildSamplingPath.m	Pooled support and quantile thinning for X_1 .
+smoothed/buildActiveSuccessors.m	Sparse active successor sets and renormalization.
+general/forwardEliminationGeneral.m	General graph importance-sampling/tabulated-support engine; Smoothed active-set branch.
runNFISAMMethod.m	NF-iSAM wrapper; batch/replay, posterior draws, KDE-style estimator packaging.
+nfisam/incrementalUpdate.m	Bayes tree rebuild, cache/reuse, leaf-to-root training.
+nfisam/combineChildSeparators.m	Sample-based child separator density combination; partial-overlap warning.
+nfisam/conditionalSamplerTrainer.m	Packs O,S,F samples and trains conditional sampler.
+flows/trainFlow.m	Custom MATLAB flow training loop and stopping rule.
commonMethodConfig.m	All budgets and provenance; NF-iSAM training cap and surface mode defaults.

13. Appendix: Minimal Patch Sketches

These are not complete patches; they are precise intent sketches for the AI.

Patch sketch 1: Slices wrapper wording

```
% runSlicesMethod.m: replace the current first comment line with something like:
%RUNSLICESMETHOD Slices Perspective baseline.
%   In the controlled three-node demonstrator, this path follows the paper-style
%   nested Slices estimator. In general-engine cases, the method uses the
%   project implementation generalization documented in result.implementation.
```

Patch sketch 2: Smoothed scope metadata

```
% evaluateSurfaceRecursion.m: add explicit scope in info
info.scope = "finite_support_one_level_rcs";
info.paperFidelity = "proposed_extension_prototype";
info.warning = "Current implementation computes the one-level Eq.(16)-style surface, not an arbitrary-
depth R_r tower.";
```

Patch sketch 3: active-set safety metadata

```
% buildActiveSuccessors.m / evaluateSurfaceRecursion.m: expose retained mass
info.retainedMassMin = min(activeInfo.retainedMass);
info.retainedMassMean = mean(activeInfo.retainedMass);
if info.retainedMassMin < 0.95
    info.warnings(end+1) = "Sparse active set retained low transition mass; RCS surface is a truncated
conditional expectation.";
end
```

Patch sketch 4: NF-iSAM deviation metadata

```
% runNFISAMMethod.m or incrementalUpdate.m: record non-paper-exact reuse semantics
result.implementation.deviationList = [
    "Bayes tree rebuilt; flow reuse based on clique/subtree identity key",
    "Child separator densities propagated through samples/resampling",
    "MATLAB-native RQ-spline flow trainer and app-runtime iteration cap"
];
```

14. References / Source Basis

- A Slices Perspective for Incremental Nonparametric Inference in High Dimensional State Spaces, Shienman, Levy-Or, Kaess, Indelman, IROS 2024. Use especially Eq. (11)-(13), Lemma 1, and the example around Eq. (16).
- Incremental Non-Gaussian Inference for SLAM Using Normalizing Flows, Huang et al., IEEE Transactions on Robotics 2023. Use especially Bayes-tree clique conditional formulation and Algorithms N1-N3.
- Project files under /mnt/data/proj_work/src, especially the wrappers, +slices, +smoothed, +nfisam, +flows, and commonMethodConfig.m.
- Project instructions under /mnt/data/proj_work/Instructions, especially smoothed_slices_exact_implementation.pdf, nf_isam_exact_implementation.pdf, slices_perspective_exact_implementation.pdf, and smoothed_slices_matlab_app_spec.pdf.